Cyberscope

A TAC Security Company

Audit Report

Hero Arena Play

May 2026

Repository https://github.com/HaGameFi/HeroArena_contracts

Commit hash 4827a7215d6876ec73d84ab69ce732810595ffcb

Audited by © cyberscope

Table of Contents

Risk Classification

The criticality of findings in Cyberscope's smart contract audits is determined by evaluating multiple variables. The two primary variables are:

1. **Likelihood of Exploitation**: This considers how easily an attack can be executed, including the economic feasibility for an attacker.
2. **Impact of Exploitation**: This assesses the potential consequences of an attack, particularly in terms of the loss of funds or disruption to the contract's functionality.

Based on these variables, findings are categorized into the following severity levels:

1. **Critical**: Indicates a vulnerability that is both highly likely to be exploited and can result in significant fund loss or severe disruption. Immediate action is required to address these issues.
2. **Medium**: Refers to vulnerabilities that are either less likely to be exploited or would have a moderate impact if exploited. These issues should be addressed in due course to ensure overall contract security.
3. **Minor**: Involves vulnerabilities that are unlikely to be exploited and would have a minor impact. These findings should still be considered for resolution to maintain best practices in security.
4. **Informative**: Points out potential improvements or informational notes that do not pose an immediate risk. Addressing these can enhance the overall quality and robustness of the contract.

Severity	Likelihood / Impact of Exploitation
Critical	Highly Likely / High Impact
Medium	Less Likely / High Impact or Highly Likely/ Lower Impact
Minor / Informative	Unlikely / Low to no Impact

Review

Audit Updates

Initial Audit	25 May 2026

Source Files

Filename	SHA256
HeroArenaMeetTheCouncil.sol	91e9390c7dbb1adbe41d3657344d642f20b05c30c1736e296e433f0cf0dcf871
HeroArenaChallenges.sol	6efdf96a632636f941b286b2df0526a535e751009dd05588bdafbc520198e6a2

Overview

`HeroArenaChallenges` and `HeroArenaMeetTheCouncil` implement Hero Arena's PvE challenge submission and reward-point flow. `HeroArenaChallenges` stores challenge completions, level metadata, and level reward points, while `HeroArenaMeetTheCouncil` acts as the coordinating contract that initializes supported levels, gates whether submissions are open, submits user completions, reads the configured reward points, and credits those points into the profile system.

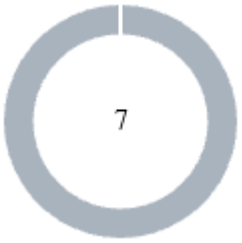
HeroArenaChallenges.sol

`HeroArenaChallenges` tracks PvE challenge completions by user and level. A challenge admin can submit a user's completion for a level, ensuring the same user can only submit each level once, and the contract records a generated challenge ID mapped to the submitted level ID. It also stores level names and reward-point values, exposes reward and submission-status views, and provides batch lookup helpers for level IDs and level metadata.

HeroArenaMeetTheCouncil.sol

`HeroArenaMeetTheCouncil` connects the challenge registry with the profile points system. The owner can initialize a fixed set of council levels and toggle whether submissions are available, while addresses with `OPERATOR_ROLE` can submit a user's completed level. On submission, the contract records the challenge through `HeroArenaChallenges`, reads the level's reward points, and forwards those points to `HeroArenaProfile` for the user.

Findings Breakdown



Critical	0
Medium	0
Minor / Informative	7

Severity	Unresolved	Acknowledged	Resolved	Other
Critical	0	0	0	0
Medium	0	0	0	0
Minor / Informative	7	0	0	0

Diagnostics

Critical Medium Minor / Informative

Severity	Code	Description	Status
●	ALI	Ambiguous Level ID	Unresolved
●	CRM	Challenge Role Misdocumented	Unresolved
●	MRB	Manual Role Bootstrap	Unresolved
●	MC	Missing Check	Unresolved
●	MEE	Missing Events Emission	Unresolved
●	MM	Multi-Authority Maintenance	Unresolved
●	ORD	Ownership Role Desync	Unresolved

ALI - Ambiguous Level ID

Criticality	Minor / Informative
Location	contracts/HeroArenaChallenges.sol#L75
Status	Unresolved

Description

`getLevelIdBatch()` returns `_lvIds[challengeId]` directly without checking whether each challenge ID exists. For non-existent challenge IDs, the mapping returns the default value `0`. If level ID `0` is valid, callers cannot distinguish a real challenge assigned to level `0` from an invalid challenge ID.

This can cause frontends, reward systems, or external integrations to treat non-existent challenges as valid level `0` completions if they rely on this batch lookup for validation or display.

```
mapping(uint256 => uint8) private _lvIds;

function submit(address _to, uint8 _lvId) external onlyChallengeAdmin
returns (uint256) {
    require(!_lvSubmits[_to][_lvId], "User can only submit once");
    _lvSubmits[_to][_lvId] = true;
    _challengeCounter += 1;
    uint256 _newChallengeId = _challengeCounter;
    _lvIds[_newChallengeId] = _lvId;
    lvCount[_lvId] += 1;
    return _newChallengeId;
}

function getLevelIdBatch(uint256[] calldata _challengeIds) external view
returns (uint8[] memory) {
    uint8[] memory _Ids = new uint8[](_challengeIds.length);
    for (uint256 i = 0; i < _challengeIds.length; i++) {
        _Ids[i] = _lvIds[_challengeIds[i]];
    }
    return _Ids;
}
```

Recommendation

Validate challenge existence before returning level IDs, or reserve level ID `0` as an invalid sentinel value. Batch lookup functions should make invalid challenge IDs explicit so consumers cannot confuse missing challenges with valid level metadata.

CRM - Challenge Role Misdocumented

Criticality	Minor / Informative
Location	contracts/HeroArenaMeetTheCouncil.sol#41
Status	Unresolved

Description

The `initLevels()` documentation states that it must be called after `HeroArenaChallenges` ownership is transferred to `HeroArenaMeetTheCouncil`. However, `HeroArenaChallenges.setLevelNameAndRewardPoints()` is not protected by ownership. It is protected by `CHALLENGE_ADMIN_ROLE`.

This can lead to incorrect deployment or operations guidance. Transferring ownership alone will not allow `initLevels()` to succeed unless `HeroArenaMeetTheCouncil` is also granted `CHALLENGE_ADMIN_ROLE` on `HeroArenaChallenges`.

```
bytes32 public constant CHALLENGE_ADMIN_ROLE =
keccak256("CHALLENGE_ADMIN_ROLE");

modifier onlyChallengeAdmin() {
    require(hasRole(CHALLENGE_ADMIN_ROLE, msg.sender), "Not a challenge
admin role");
    _;
}

function setLevelNameAndRewardPoints(uint8 _lvId, string calldata
_lvName, uint256 _lvPts)
    external
    onlyChallengeAdmin
{
    _lvNames[_lvId] = _lvName;
    _lvRewardPoints[_lvId] = _lvPts;
}

/**
 * Initialize level names and points. Must be called after
HeroArenaChallenges
 * ownership has been transferred to this contract.
 */
function initLevels() external onlyOwner {
    HeroArenaChallengesSC.setLevelNameAndRewardPoints(0, "Ladder Climb",
```

```
5);
    ...
}
```

Recommendation

Update the deployment comments and initialization documentation to reference `CHALLENGE_ADMIN_ROLE` instead of ownership transfer. Deployment scripts should explicitly grant the council contract the required role before calling `initLevels()`.

MRB - Manual Role Bootstrap

Criticality	Minor / Informative
Location	contracts/HeroArenaChallenges.sol#L7,L27,L32,L39,L52
Status	Unresolved

Description

`HeroArenaChallenges` defines `CHALLENGE_ADMIN_ROLE` and uses it to restrict challenge submission and level configuration. However, the constructor only grants `DEFAULT_ADMIN_ROLE` to the deployer and does not grant `CHALLENGE_ADMIN_ROLE` to any address.

As a result, the core admin functions are unusable immediately after deployment until the deployer manually grants the challenge admin role. This creates a deployment footgun where challenge setup and submissions fail if the manual post-deployment role assignment is missed.

```
bytes32 public constant CHALLENGE_ADMIN_ROLE =
keccak256("CHALLENGE_ADMIN_ROLE");

modifier onlyChallengeAdmin() {
    require(hasRole(CHALLENGE_ADMIN_ROLE, msg.sender), "Not a challenge
admin role");
    _;
}

constructor() {
    _grantRole(DEFAULT_ADMIN_ROLE, msg.sender);
}

function submit(address _to, uint8 _lvId) external onlyChallengeAdmin
returns (uint256) {
    ...
}

function setLevelNameAndRewardPoints(uint8 _lvId, string calldata
_lvName, uint256 _lvPts)
    external
    onlyChallengeAdmin
{
```

```
    ...
}
```

Recommendation

Grant the challenge admin role during deployment to the intended administrator, or accept an explicit admin address in the constructor and assign all required bootstrap roles there. Deployment scripts should also verify that required operational roles are assigned before the contract is considered initialized.

MC - Missing Check

Criticality	Minor / Informative
Location	contracts/HeroArenaChallenges.sol#L39 contracts/HeroArenaMeetTheCouncil.sol#L30
Status	Unresolved

Description

The contract is processing variables that have not been properly sanitized and checked that they form the proper shape. These variables may produce vulnerability issues. More specifically, the `_to` address isn't checked if it's the zero address and the `_lvlId` isn't checked whether it's a valid level. As well as the addresses of the contracts aren't checked in the constructor.

```
function submit(address _to, uint8 _lvId) external onlyChallengeAdmin
returns (uint256) {
    require(!_lvSubmits[_to][_lvId], "User can only submit once");
    _lvSubmits[_to][_lvId] = true;
    _challengeCounter += 1;
    uint256 _newChallengeId = _challengeCounter;
    _lvIds[_newChallengeId] = _lvId;
    lvCount[_lvId] += 1;
    return _newChallengeId;
}

constructor(HeroArenaChallenges _HeroArenaChallengesSC, HeroArenaProfile
_HeroArenaProfileSC) Ownable(msg.sender) {
    HeroArenaChallengesSC = _HeroArenaChallengesSC;
    HeroArenaProfileSC = _HeroArenaProfileSC;

    _grantRole(DEFAULT_ADMIN_ROLE, msg.sender);
}
```

Recommendation

The team is advised to properly check the variables according to the required specifications.

MEE - Missing Events Emission

Criticality	Minor / Informative
Location	contracts/HeroArenaChallenges.sol#L52
Status	Unresolved

Description

The contract performs actions and state mutations from external methods that do not result in the emission of events. Emitting events for significant actions is important as it allows external parties, such as wallets or dApps, to track and monitor the activity on the contract. Without these events, it may be difficult for external parties to accurately determine the current state of the contract.

```
function setLevelNameAndRewardPoints(uint8 _lvId, string calldata
_lvName, uint256 _lvPts) external onlyChallengeAdmin {
    _lvNames[_lvId] = _lvName;
    _lvRewardPoints[_lvId] = _lvPts;
}
```

Recommendation

It is recommended to include events in the code that are triggered each time a significant action is taking place within the contract. These events should include relevant details such as the user's address and the nature of the action taken. By doing so, the contract will be more transparent and easily auditable by external parties. It will also help prevent potential issues or disputes that may arise in the future.

MM - Multi-Authority Maintenance

Criticality	Minor / Informative
Location	contracts/HeroArenaMeetTheCouncil.sol contracts/HeroArenaChallenges.sol contracts/HeroArenaProfile.sol contracts/HeroArenaMiningFactoryV1.sol contracts/HapToken.sol contracts/HapVesting.sol contracts/HapTreasury.sol
Status	Unresolved

Description

The system uses multiple authority models across contracts, including `Ownable`, `AccessControl`, role-gated operators, owner-gated factories, and cross-contract role dependencies. Several flows depend on manually maintained permissions, such as granting `CHALLENGE_ADMIN_ROLE` to the council contract, keeping ownership aligned with `DEFAULT_ADMIN_ROLE`, assigning profile point/operator roles, and preserving ownership over deployed avatar contracts.

This creates operational fragility because contracts may deploy successfully but remain partially non-functional until the correct role and ownership wiring is completed. Future ownership transfers, role revocations, or missed deployment steps can break core flows even when the underlying contracts are otherwise working.

```
contract HeroArenaProfile is AccessControl, ERC721Holder, Ownable {
    bytes32 public constant AVATAR_ROLE = keccak256("AVATAR_ROLE");
    bytes32 public constant FRAME_ROLE = keccak256("FRAME_ROLE");
    bytes32 public constant POINT_ROLE = keccak256("POINT_ROLE");
    bytes32 public constant SPECIAL_ROLE = keccak256("SPECIAL_ROLE");

    constructor(...) Ownable(msg.sender) {
        ...
        _grantRole(DEFAULT_ADMIN_ROLE, msg.sender);
    }
}

contract HeroArenaMeetTheCouncil is AccessControl, Ownable {
    bytes32 public constant OPERATOR_ROLE = keccak256("OPERATOR_ROLE");

    constructor(...) Ownable(msg.sender) {
        ...
        _grantRole(DEFAULT_ADMIN_ROLE, msg.sender);
    }

    function submitLv(address _userAddress, uint8 _lvId) external
onlyOperator {
        uint256 _challengeId = HeroArenaChallengesSC.submit(_userAddress,
_lvId);
        HeroArenaProfileSC.increaseUserPoints(_userAddress,
_lvRewardPoints, _lvId);
    }
}

contract HeroArenaChallenges is AccessControl {
    bytes32 public constant CHALLENGE_ADMIN_ROLE =
keccak256("CHALLENGE_ADMIN_ROLE");

    modifier onlyChallengeAdmin() {
        require(hasRole(CHALLENGE_ADMIN_ROLE, msg.sender), "Not a
challenge admin role");
        _;
    }
}
```

Recommendation

Define and document a single authority model or a complete role-management runbook for the system. Deployment and ownership-transfer procedures should atomically assign required roles, revoke stale authorities, and verify cross-contract permissions before

contracts are considered operational. Where possible, reduce manual wiring by granting required roles during construction or through explicit initialization functions.

ORD - Ownership Role Desync

Criticality	Minor / Informative
Location	contracts/HeroArenaMeetTheCouncil.sol#L30
Status	Unresolved

Description

`HeroArenaMeetTheCouncil` inherits both `Ownable` and `AccessControl`. The constructor grants `DEFAULT_ADMIN_ROLE` to the initial owner, but the contract does not override ownership transfer logic to keep the two authority systems synchronized. If ownership is transferred, the new owner receives `onlyOwner` permissions but does not automatically receive `DEFAULT_ADMIN_ROLE`.

This can leave the previous owner as the AccessControl admin, retaining the ability to grant or revoke `OPERATOR_ROLE`. The new owner may control owner-only functions such as level initialization and submission availability, while being unable to administer operator permissions, creating split governance and stale privileged access.

```
contract HeroArenaMeetTheCouncil is AccessControl, Ownable {
    bytes32 public constant OPERATOR_ROLE = keccak256("OPERATOR_ROLE");

    modifier onlyOperator() {
        require(hasRole(OPERATOR_ROLE, msg.sender), "Not an operator
role");
        _;
    }

    constructor(HeroArenaChallenges _HeroArenaChallengesSC,
HeroArenaProfile _HeroArenaProfileSC)
        Ownable(msg.sender)
    {
        HeroArenaChallengesSC = _HeroArenaChallengesSC;
        HeroArenaProfileSC = _HeroArenaProfileSC;

        _grantRole(DEFAULT_ADMIN_ROLE, msg.sender);
    }
}
```

Recommendation

Keep ownership and role administration aligned during ownership changes. Either use a single authority model consistently, or update ownership transfer handling so the new owner receives `DEFAULT_ADMIN_ROLE` and the previous owner's role is revoked when appropriate.

Functions Analysis

Contract	Type	Bases		
	Function Name	**Visibility**	**Mutability**	**Modifiers**
HeroArenaMeetTheCouncil	Implementation	AccessControl, Ownable		
		Public	✓	Ownable
	initLevels	External	✓	onlyOwner
	updateAvailableSubmit	External	✓	onlyOwner
	submitLv	External	✓	onlyOperator
HeroArenaChallenges	Implementation	AccessControl		
		Public	✓	-
	submit	External	✓	onlyChallengeAdmin
	setLevelNameAndRewardPoints	External	✓	onlyChallengeAdmin
	getLevelRewardPoints	External		-
	getSubmitStatus	External		-
	getLevelIdBatch	External		-
	getLevelNameAndPointsBatch	External		-

Summary

Hero Arena contracts implement a PvE challenge suite built around level configuration, one-time user challenge submissions, reward-point assignment, and an operator-driven council submission flow. This audit investigates security issues, business logic concerns and potential improvements.

Disclaimer

The information provided in this report does not constitute investment, financial or trading advice and you should not treat any of the document's content as such. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes nor may copies be delivered to any other person other than the Company without Cyberscope's prior written consent. This report is not nor should be considered an "endorsement" or "disapproval" of any particular project or team. This report is not nor should be regarded as an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Cyberscope to perform a security assessment. This document does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors' business, business model or legal compliance. This report should not be used in any way to make decisions around investment or involvement with any particular project. This report represents an extensive assessment process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk Cyberscope's position is that each company and individual are responsible for their own due diligence and continuous security Cyberscope's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies and in no way claims any guarantee of security or functionality of the technology we agree to analyze. The assessment services provided by Cyberscope are subject to dependencies and are under continuing development. You agree that your access and/or use including but not limited to any services reports and materials will be at your sole risk on an as-is where-is and as-available basis Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives false negatives and other unpredictable results. The services may access and depend upon multiple layers of third parties.

About Cyberscope

Cyberscope is a TAC blockchain cybersecurity company that was founded with the vision to make web3.0 a safer place for investors and developers. Since its launch, it has worked with thousands of projects and is estimated to have secured tens of millions of investors' funds.

Cyberscope is one of the leading smart contract audit firms in the crypto space and has built a high-profile network of clients and partners.

A **TAC Security** Company

The Cyberscope team

cyberscope.io